

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/334077747>

Test Program Generator MicroTESK for RISC-V

Conference Paper · December 2018

DOI: 10.1109/MTV.2018.00011

CITATIONS

7

READS

707

6 authors, including:



Mikhail Chupilko

Russian Academy of Sciences

29 PUBLICATIONS 146 CITATIONS

SEE PROFILE



Alexander Kamkin

Russian Academy of Sciences

53 PUBLICATIONS 254 CITATIONS

SEE PROFILE



Alexandr S. Protsenko

Russian Academy of Sciences

10 PUBLICATIONS 25 CITATIONS

SEE PROFILE



Sergey Smolov

Russian Academy of Sciences

22 PUBLICATIONS 78 CITATIONS

SEE PROFILE

Test Program Generator MicroTESK for RISC-V

Mikhail Chupilko¹, Alexander Kamkin^{1,2,3,4}, Artem Kotsyniak¹,
Alexander Protsenko¹, Sergey Smolov¹, Andrei Tatarnikov⁴

¹ Ivannikov Institute for System Programming of the Russian Academy of Sciences (ISP RAS)

² Lomonosov Moscow State University (MSU)

³ Moscow Institute of Physics and Technology (MIPT)

⁴ National Research University – Higher School of Economics (HSE)

Email: {chupilko, kamkin, kotsyniak, protsenko, smolov, andrewt}@ispras.ru

Abstract—The paper presents a test program generator for functional verification of RISC-V microprocessors. The generator is implemented on the base of MicroTESK framework and consists of formal specifications of RISC-V ISA and ISA-independent core. The specifications describe instructions' syntax and semantics and can be easily modified to support more instructions (including custom extensions). The core implements techniques of instruction sequences composition and test data generation. Test programs are generated from test templates, describing the programs' structural and behavioral properties; among generation techniques, random, combinatorial, and constraint-based ones are supported.

I. INTRODUCTION

RISC-V is an open RISC-architecture [1] developed by UC Berkeley in 2010. Nowadays, RISC-V Foundation [2] defines the ways of its further development; RISC-V ISA and all standard extensions are also issued there. Key features of RISC-V architecture is its openness and extendibility. Among the companies working under development of RISC-V microprocessors, NVIDIA [3] and Samsung [4] are distinguished. Also, ETH Zurich and University of Bologna [5] conduct common research in this field.

It is still impossible to design a microprocessor without bugs, that is why the task of microprocessor verification has been of high priority for many years, and a certain toolset is also required during development of RISC-V microprocessors. The following approach to verification is widely distributed: test programs are executed on RTL model of microprocessor, and the execution traces are compared against the reference simulator [6]. The test programs are created by means of Test Program Generators (TPGs). The important requirement for RISC-V TPGs is an easy adaptation for different versions of the architecture, including different extension sets. We think that the most suitable solution should be based on formal specifications [7] of ISA. In this case, instruction description is separated from test generation logics and can be modified without changing of the generator core.

In this work, MicroTESK for RISC-V TPG [8] is considered, developed by means of MicroTESK (Microprocessor TEsting and Specification Kit) [9], [10]. The generator consists of two parts: architecture-independent core (library components) and formal specifications of RISC-V instructions in the nML language [11]. Such a subdivision of the generation

parts decreases labor efforts required for their development and facilitates the process of adaptation to architecture changes.

The rest of the paper is organized as follows. Section II reviews the related work in the field of test program generation for microprocessors. Section III overviews the MicroTESK TPG for RISC-V. It contains basic principles of the proposed approach and features of its application to the RISC-V architecture. Section IV describes the proposed generator, including supported instructions, volume of specifications, labor efforts. Section V concludes the paper and outlines future research directions.

II. RELATED WORK

Together with the RISC-V ISA, a test set for RISC-V instruction implementations has been created in UC Berkeley [12]. It is oriented to check each instruction individually without explicit covering such situations that are specific for the current architecture. To check different variants of instructions pipeline co-execution, a generator named RISC-V Torture Test Generator (or simply Torture) [13] has been also developed. It constructs test programs randomly by combining short user-defined instruction sequences and assigning automatically their register numbers.

There are universal TPGs, that can be applied to different architectures. The best known such a TPG is Genesys-Pro [7] from IBM Research. The generator uses the following input data: microprocessor model and test templates, describing instruction sequences and selecting probability distributions and constraints for them. During the generation phase, this TPG predicts register and memory states by execution of the constructed code on some external simulator. This scheme is also used for obtaining self-checks for tests.

Another generator named RAVEN (Random Architecture Verification Machine) [14] has been developed by Obsidian Software (now owned by ARM). This tool is based on the same principles as Genesys-Pro; its generation logics is separated by architecture describing model. RAVEN generates programs on the base of test templates and can create random and targeted tests. Like Genesys-Pro, RAVEN uses an external simulator to get correct microprocessor state.

Reviewed above TPGs have some disadvantages. Torture tool is oriented to random generation only, not being able to construct tests according to used-defined test scenarios.

Also, to extend the list of supported instructions, the generator source code has to be modified. RAVEN and Genesys-Pro tools are free from these problems, but they do not support RISC-V, to say nothing of their being a proprietary tools, making it impossible to add this support manually. Having supported by large corporations, they are hardly oriented to particular requests of third-parties development projects. The last couple of things we have to mention is that, as far as we know, the average speed of instruction generation in them is quite low (not more than dozens of instructions per second) and scenarios of test generation in them lack readability and reusability.

The approach implemented in MicroTESK is a fast (hundreds and thousands instructions per second) open-source solution with object-oriented test scenarios (using Ruby), aimed to facilitate development and support of TPG by extraction all the necessary input information from just one source: formal specifications of ISA. Formal representation of ISA seems to be a prospective solution, opening the way to formal verification and to automated obtaining some development tools (like simulators, compilers, disassemblers, and high-level synthesizers), while taking not more efforts than support of an architecture model of ISA.

III. MICROTesk FOR RISC-V OVERVIEW

A. MicroTESK Toolkit

MicroTESK for RISC-V test program generation is based on MicroTESK Toolkit [9], [10]. It includes two main parts: the modeling framework and the testing framework. The former is responsible for construction of microprocessor architecture model by processing its formal specifications; the latter allows generation of test programs based on the given architecture model and test templates. The MicroTESK architecture is shown in Figure 1.

The input data for the modeling framework are formal specifications of ISA written in nML [11]. It implements analyzers and constructors for microprocessor model obtaining. The obtained model consists of the following parts:

- 1) the metadata that holds the catalogue of supported registers and instructions;
- 2) the instruction set simulator (ISS) that executes instructions to track current register and memory states during generation;
- 3) the coverage model that holds possible instruction execution paths.

The testing framework receives test templates developed in the Ruby-based language [15]. The language supports description of abstract test scenarios, where concrete instructions, their order and operands are not fixed. The concretizing is made dynamically depending on generation parameters. In the templates, it is possible to use registers and call instructions using Assembler-like syntax. All the language commands required for the current architecture are created dynamically basing on the metadata. The test templates are developed by verification engineers, pursuing selected verification targets.

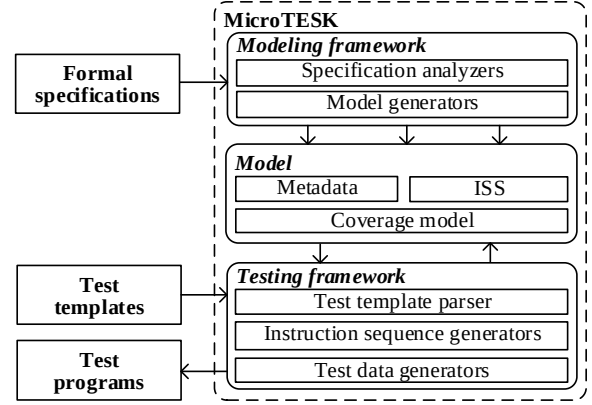


Fig. 1. Architecture of the MicroTESK TPG

Development of some test types can be automated by means of the MicroTESK out-of-the-box:

- 1) tests for separated instructions, using random operand values;
- 2) tests for separated instructions, using boundary operand values;
- 3) tests for separated instructions, covering all execution paths described in specifications;
- 4) tests for short chains of instructions, resulting as enumeration of all instructions of different types.

The template-based process of test program generation consist of the following stages:

- 1) parsing the test template to construct internal representation;
- 2) constructing abstract instruction sequences (without concrete test data);
- 3) for each of them the following actions are applied:
 - a) concretizing of instructions:
 - selection of registers;
 - generation of test data;
 - creation of the initialization code;
 - b) executing the instructions in the internal MicroTESK simulator;
 - c) when creating self-check tests: insertion the self checks based on ISS into the test cases;
 - d) printing the resulting instructions to an assembly file.

B. Instruction set specification

RISC-V architecture is described in "The RISC-V Instruction Set Manual" [16], [17]. ISA consists of basic instruction subset and extensions. A RISC-V microprocessor can be implemented with 32-, 64-, or 128-bit address space and registers. The following instruction subsets are defined for 32- and 64-bit versions:

- 1) RV32I (basic subset) contains integer arithmetic, branching, memory access, and system calls, common for both 32- and 64-bit versions;

- 2) RV64I (built upon RV32I) contains additional integer arithmetic instructions and memory access instructions for 64-bit version;
- 3) RV32M (standard extension) contains integer multiplication and division instructions, common for both 32- and 64-bit versions;
- 4) RV64M (built upon RV32M) contains additional integer multiplication and division instructions for 64-bit version;
- 5) RV32A (standard extension) contains atomic reading-writing-modifying memory instructions for both 32- and 64-bit versions;
- 6) RV64A (built upon RV32A) contains additional memory instructions for 64-bit version;
- 7) RV32F (standard extension) contains single-precision floating-point arithmetic and conversion instructions, common for both 32- and 64-bit versions;
- 8) RV64F (built upon RV32F) contains additional conversion instructions for 64-bit version;
- 9) RV32D (standard extension) contains double-precision floating-point arithmetic and conversion instructions, common for both 32- and 64-bit versions;
- 10) RV64D (built upon RV32D) contains additional conversion instructions for 64-bit version;
- 11) RV32C (standard extension) contains compressed (16-bit) instructions, common for both 32- and 64-bit versions;
- 12) RV64C (built upon RV32C) contains additional compressed (16-bit) instructions for 64-bit version;
- 13) privileged system instructions.

The following control registers hold the current state of a RISC-V microprocessor:

- 1) instruction counter PC (32 or 64 bits);
- 2) 32 general-purpose registers (GPRs) x0-x31 (32 bits wide for RV32 and 64 bits wide for RV64);
- 3) 32 floating-point single-precision registers f0-f31 (in RV32F and RV64F);
- 4) 32 floating-point double-precision registers f0-f31 (in RV32D and RV64D);
- 5) 32 bits wide register FCSR (Floating-Point Control and Status Register) (in RV32F, RV64F, RV32D, and RV64D);
- 6) 32 bits wide (for RV32) or 64 bits wide (for RV64) registers CSR (Control and Status Register) (up to 4096).

RISC-V-compatible microprocessors supports three privilege levels. To shift between them, some system instructions should be used.

- 1) M (Machine) – the highest privilege level;
- 2) S (Supervisor) – operation system level;
- 3) U (User) – user application level.

All mentioned instruction subsets have been described in nML specifications. Architecture version (32 or 64-bit) is selected by an input parameter.

Let us see how nML specifications look like on the following example (see Figure 2), including definition of GPRs

```
// Current word length (it is up to configuration)
let XLEN = #ifdef RV64I 64 #else 32 #endif
// card means unsigned int, XLEN is its bit size
type XWORD = card(XLEN)
// GPRs
reg XREG [32, XWORD]
// Addressing modes for GPRs
mode X (i: card(5)) = XREG [i]
  syntax = format("x%d", i) // Assembler representation
  image = format("%5s", i) // Binary representation
// This instruction adds register's value with
// a 12-bit immediate
op addi(rd: X, rs1: X, imm: card(12))
  syntax = format("addi_%%s, %%s, 0x%%x",
    rd.syntax, rs1.syntax, imm)
  image = format("%12s%%s000%%s0010011",
    imm, rs1.image, rd.image)
  action = { // The action itself
    rd = rs1 + sign_extend(XWORD, imm);
  }
```

Fig. 2. nML representation of GPRs and ADDI instruction

```
op div(rd: X, rs1: X, rs2: X)
  syntax = format("div_%%s, %%s, %%s",
    rd.syntax, rs1.syntax, rs2.syntax)
  image = format("0000001%%s100%%s0110011",
    rs2.image, rs1.image, rd.image)
  action = {
    if rs2 == 0 then
      mark('div_by_zero');
      rd = -1;
    elif rs1 == 1 << (XLEN - 1) && rs2 == -1 then
      mark('min_xint_negation');
      rd = rs1;
    else
      mark('normal');
      rd = cast(XINT, rs1) / cast(XINT, rs2);
    endif;
  }
```

Fig. 3. nML representation of DIV instruction

together with addressing modes to access these registers, and an instruction of addition with immediate value. Not going to define the nML language in a formal way, we will just mention that objects with type identifier *op* (meaning operation or instruction) have three basic properties: the one for Assembler syntax, the one for binary representation, and the one for the description of what this particular instruction should do (sub-procedure calls are allowed).

Each instruction can have different execution paths. The example given in Figure 3 demonstrates specification of integer division instruction *DIV*. For each execution paths, MicroTESK constructs constraints (in form of formulas), allowing to cover these paths. Operator *mark* allows naming these constraints (see example below) to refer them from test templates.

C. Test program generation

After all specifications having been developed and the architecture model having been automatically created, it is possible to use MicroTESK's testing framework to construct test programs. To do so, the tool utilizes test templates in Ruby language, describing testing scenarios by means of certain constructions.

```

class RandomTemplate < RISCvBaseTemplate
  # Main method of the test template
  def run
    # Probability distribution for input data
    int_dist = dist(
      range(:value => 0, :bias => 25),
      range(:value => 1..2, :bias => 25),
      range(:value => 0xffffffff..0xffffffff, :bias => 50))
    # Description of stimulus structure
    sequence {
      add x(_), x(_), x(_) do
        # Definition of test data generation method
        testdata('random_biased', :dist => int_dist)
      end
    }.run 10 # The number of test cases
  end
end

```

Fig. 4. Example of test template in Ruby

```

# Declares the product combinator
sequence(:data_combinator => 'product') {
  # Sets up the first instruction in the sequence
  add x8, x9, x10 do
    testdata('range', :min => 1, :max => 3)
  end
  # The same with the second instruction
  sub x11, x12, x13 do
    testdata('range', :min => 1, :max => 3)
  end
end
}.run

```

Fig. 5. Test sequence generation by product combinator

When constructing test programs, different techniques are used: randomization, enumeration, constraint solving. E.g., the test template given in Figure 4 results in 10 test cases, each of them consisting of ADD instruction with random register numbers.

The test template part given in Figure 5 declares a sequence of instructions ADD and SUB, receiving as input data a product of values from the range [1, 3]. It will result in generation of 81 sequences in total (three power four, where three is the number of different possible values, and four is the number of input registers).

The example given in Figure 6 demonstrates a co-application of combinatorial generation and constraint-based generation. The input parameters for DIV instructions are constructed in the way to cover all execution paths given in the specification. Each DIV selection is supplemented by SLT and MUL instruction (only one of them for the current sequence), that gives 6 sequences.

Each test sequence is supported by initialization code that relocates test data into registers and memory. So-called preparators are used for that. It is possible to define several preparators that will be used accordingly to the test data format. E.g., the preparators writing data into general purpose registers are given in Figure 7.

IV. CHARACTERISTICS OF THE DEVELOPED TPG

MicroTESK for RISC-V supports generation of the following tests:

- 1) random tests:

```

# Combines results of nested blocks
block(:combinator => 'product', :compositor => 'random'){
  iterate { # Iterates nested instructions
    div x8, x9, x10 do situation('div_by_zero') end
    div x8, x9, x10 do situation('min_xint_negation') end
    div x8, x9, x10 do situation('normal') end
  }
  iterate { # Iterates nested instructions
    slt x(_), zero, x8
    mul x(_), x8, x10
  }
}.run

```

Fig. 6. Test sequence generation in combinatorial and constraint-based ways

```

# Default preparator for X registers
preparator(:target => 'X') {
  # Writes the value into the target register
  li target, value
}
# Preparator for the case when value is equal to 0
preparator(:target => 'X', :mask => '0000_0000') {
  or target, zero, zero
}

```

Fig. 7. Test data preparation

- a) randomization of instruction sequences;
- b) randomization of test data;
 - i) selection of probability distribution;
 - ii) random choice of values from the ranges;
- 2) combinatorial tests:
 - a) combination of instruction sequences;
 - b) enumeration of test data:
 - i) enumeration of boundary values;
 - ii) enumeration of values from the ranges;
 - c) combinatorial tests for branch instructions:
 - i) enumeration of control flow graphs;
 - ii) enumeration of execution traces;
 - d) tests covering instruction execution paths:
 - i) enumeration of execution paths;
 - ii) constraint solving;
 - e) self-check tests;
 - f) tests including features of test types mentioned above.

MicroTESK for RISC-V includes test templates demonstrating features of the generator. It also contains test templates corresponding to the UC Berkeley's test set [12].

The task of development of formal RISC-V specifications required four man-months (conducted by two experienced verification engineers). Totally, 262 instructions have been described (including 59 pseudo instructions, used by RISC-V Assembler). Specification volume is about 3500 lines of code. The table I contains information on the specified instructions.

Possible nML-specifications mistakes and bugs manifest either by an incorrect Assembler instruction format or in form of incorrect results of execution of instructions on the internal simulator. Thus, test programs that are generated using those

TABLE I
SPECIFIED INSTRUCTIONS OF RISC-V

Instruction subset	Description	Number of instructions
RV32I	Base integer arithmetic, memory access, branches	37
RV32I	Access to control registers	6
RV32I	System instructions	9
RV64I	Base integer arithmetic, branches	12
RV32M	Integer multiplication and division	8
RV64M	Integer multiplication and division	5
RV32A	Atomic instructions	11
RV64A	Atomic instructions	11
RV32F	Floating-point instructions	26
RV64F	Floating-point instructions	4
RV32D	Floating-point instructions	26
RV64D	Floating-point instructions	4
RV32C	Compressed instructions	35
RV64C	Compressed instructions	7
Pseudo	Pseudo instructions	59
Totally		262

incorrectly specified instructions cannot be compiled or lead to incorrect predicted results of their execution, especially if there are branching instructions or load immediate instructions among them, on which the construction of test program's execution paths is based. To reveal the specification bugs, the following actions have been applied for all test templates included into MicroTESK for RISC-V:

- 1) Test programs were generated according to the test templates. Generated test programs were executed in ISS, producing execution traces (to account instruction executions, register or memory access and other events).
- 2) The test programs were compiled and executed on external simulators QEMU for RISC-V [18] and Spike [19], producing execution traces of the same format.
- 3) Execution traces for the same program were compared by means of Trace Matcher [20], generating a difference report.

There is also a complementary way to improve the quality of nML specifications: using of self-check tests. In this case, only generated (with turned on self-check option) test programs and an external simulator are needed. Such test programs can, say, jump to some error code section if registers, having used in that particular test case, have values different from those predicted by ISS.

V. CONCLUSION

This paper describes a TPG for functional verification of RISC-V microprocessors, which has been developed at ISP PAS on the basis of the MicroTESK framework. The TPG supports generating various types of tests including the ones

similar to UC Berkeley's RISC-V test set [12] and the ones generated by Torture Test Generator [13]. The important advantage of MicroTESK for RISC-V is its extensibility, which allows adding support for new instruction subsets without modifying the generator core.

In the nearest future, we are planning to create formal specifications of the RISC-V MMU and of additional RISC-V ISA extensions (vector instructions and other). Also, we are planning to implement the following additional facilities of MicroTESK:

- 1) a generator of ISA-level coverage reports for the generated test programs;
- 2) a tool for automated construction of post-silicon test program generators that are loaded and run inside FPGAs or chip prototypes under verification and based on nML specifications;
- 3) a tool for static analysis and deductive verification of binary code.

ACKNOWLEDGMENT

The authors would like to thank Russian Foundation for Basic Research (RFBR). The reported study was supported by RFBR, research project №18-07-01218.

REFERENCES

- [1] *RISC-V* – <https://en.wikipedia.org/wiki/RISC-V>
- [2] *RISC-V site* – <https://riscv.org>
- [3] https://www.phoronix.com/scan.php?page=news_item&px=NVIDIA-RISC-V-Next-Gen-Falcon
- [4] <https://www.electronicweekly.com/blogs/mannerisms/dilemmas/samsung-defection-arm-risc-v-2016-11>
- [5] *PULP site* – <https://www.pulp-platform.org>
- [6] A. Kamkin. Generatsiya testovyykh programm dlya mikroprotssorov (Test Program Generation for Microprocessors). Trudy ISP RAN (Proceedings of ISP RAS), Volume 14, Part 2, 2008, P. 23-64.
- [7] A. Adir, E. Almog, L. Fournier, E. Marcus, M. Rimon, M. Vinov, A. Ziv. *Genesys-Pro: Innovations in Test Program Generation for Functional Processor Verification*. Design & Test of Computers, 2004. pp. 84–93.
- [8] *MicroTESK for RISC-V* – <https://forge.ispras.ru/projects/microtesk-riscv>
- [9] *MicroTESK* – <http://forge.ispras.ru/projects/microtesk>
- [10] M. Chupilko, A. Kamkin, A. Kotsynnyak, A. Tatarnikov. MicroTESK: Specification-Based Tool for Constructing Test Program Generators. Hardware and Software: Verification and Testing. Proceedings of 13th International Haifa Verification Conference, 2017. P. 217-220.
- [11] M. Freericks. *The nML Machine Description Formalism*. Technical Report TR SM-IMP/DIST/08, TU Berlin CS Department, 1993. 47 p.
- [12] *RISC-V Tests* – <https://github.com/riscv/riscv-tests>
- [13] *RISC-V Torture Test Generator* – <https://github.com/ucb-bar/riscv-torture>
- [14] *RAVEN TPG* – <http://www.slideshare.net/DVClub/introducing-obsidian-software-andravengcs-for-powerpc>
- [15] A. Tatarnikov. *Language for Describing Templates for Test Program Generation for Microprocessors*. Proceedings of ISP RAS, 28(4), 2016. pp. 81–102.
- [16] A. Waterman, K. Asanovic. *The RISC-V Instruction Set Manual. Volume I: User-Level ISA. Version 2.2*. University of California, Berkeley. May 7, 2017. 145 p.
- [17] A. Waterman, K. Asanovic. *The RISC-V Instruction Set Manual. Volume II: Privileged Architecture. Version 1.10*. University of California, Berkeley. May 7, 2017. 91 p.
- [18] *ISA Simulator QEMU for Verification (QEMU4V)* – <https://forge.ispras.ru/projects/qemu4v>
- [19] *RISC-V ISA Simulator Spike* – <https://github.com/riscv/riscv-isa-sim>
- [20] *Trace Matcher* – <https://forge.ispras.ru/projects/traceutils>